

UNIVERSIDAD TECNOLÓGICA NACIONAL
Tecnicatura Universitaria en Programación a Distancia

TRABAJO PRÁCTICO N.º 1

Del dominio al esquema: modelo entidad-relación, modelo relacional, normalización y DDL

Materia	Base de Datos I
Carrera	Tecnicatura Universitaria en Programación a Distancia (UTN)
Trabajo práctico	N.º 1 — Del dominio al esquema: modelo ER, modelo relacional, normalización y DDL
Proyecto integrador	Food Store (sistema de gestión de pedidos de un negocio de comidas) — pieza correspondiente a la Semana 1
Modalidad	
Fecha de entrega	

1. Presentación y objetivos

Este trabajo práctico inaugura el hilo conductor de la materia: el proyecto integrador **Food Store**, un sistema de gestión de pedidos para un negocio de comidas. A partir de un enunciado en lenguaje natural, van a recorrer el camino completo que hace una base de datos relacional bien diseñada: analizar el dominio, modelarlo con un diagrama entidad-relación, derivar el esquema relacional correspondiente, verificar (y justificar) que ese esquema está normalizado, y finalmente escribir el DDL que lo crea en PostgreSQL.

Al finalizar este trabajo van a haber producido **schema.sql**, la primera pieza del proyecto integrador, que se irá ampliando semana a semana con datos, consultas, vistas, funciones y transacciones.

Al completar este TP vas a ser capaz de:

- Construir un modelo entidad-relación a partir de un enunciado, identificando entidades, atributos, claves, cardinalidades y participación (total o parcial) de cada relación.
- Derivar el modelo relacional correspondiente a un modelo ER, resolviendo relaciones 1:N mediante clave foránea y relaciones N:M mediante una tabla intermedia (entidad asociativa).
- Aplicar el proceso de normalización hasta 3FN/BCNF sobre un conjunto de datos, identificando dependencias funcionales y justificando por qué cada forma normal se cumple o se viola.
- Escribir DDL correcto en PostgreSQL: tipos de datos apropiados, claves primarias y foráneas, restricciones declarativas (NOT NULL, UNIQUE, CHECK, DEFAULT) e índices justificados por el uso esperado de las tablas.

2. El caso: Food Store

Un negocio de comidas quiere reemplazar sus planillas de cálculo por una base de datos. Así describe su operación el propietario:

El negocio, en sus palabras: «Vendemos productos que agrupamos en categorías, como pizzas o bebidas. Los clientes se registran con sus datos y hacen pedidos. Cada pedido puede incluir varios productos, y cada producto se vende en muchos pedidos distintos. Necesitamos saber cuánto se vendió de cada línea, aunque el precio de un producto cambie con el tiempo: lo que se facturó en marzo no puede cambiar porque en abril subimos el precio.»

Además de esa descripción general, la cátedra fija las siguientes **reglas de negocio** (restricciones del dominio) que el modelo debe respetar:

- R1. Todo producto pertenece exactamente a una categoría; una categoría puede tener muchos productos, o ninguno todavía (por ejemplo, una categoría recién creada).
- R2. Todo pedido pertenece exactamente a un cliente registrado; un cliente puede no haber hecho ningún pedido aún, o haber hecho varios.
- R3. Un pedido puede incluir varios productos distintos, y un mismo producto puede aparecer en muchos pedidos distintos (relación de muchos a muchos entre pedido y producto).
- R4. De cada producto dentro de un pedido interesa registrar cuántas unidades se pidieron y a qué precio unitario, porque el precio de lista de un producto puede cambiar con el tiempo y no debe alterar pedidos ya facturados.
- R5. Un producto no puede tener stock ni precio negativos.
- R6. Cada cliente tiene un correo electrónico que lo identifica de forma única en el sistema.
- R7. El negocio no elimina físicamente productos ni categorías dados de baja: prefiere conservarlos marcados como inactivos, para no perder el historial de pedidos que los referencian.

Nota: estas reglas son la base para las cardinalidades, participaciones y restricciones que vas a modelar en cada parte. Se te pueden ocurrir atributos adicionales razonables (por ejemplo, teléfono del cliente, descripción del producto); si los agregas, déjalo aclarado en tu diccionario de datos.

3. Consigna

El trabajo se organiza en cuatro partes secuenciales: cada una toma como entrada el resultado de la anterior. Te recomendamos resolverlas en orden.

Parte 1 — Modelo entidad-relación (25%)

A partir del enunciado de la Sección 2, construí el diagrama entidad-relación completo del dominio **Food Store**. Puedes usar notación Chen o Pata de Gallo (Crow's Foot); elegí una y sostenerla en todo el diagrama.

Tu diagrama debe incluir, para cada entidad:

- Nombre de la entidad y su clave primaria (*marcada de forma inequívoca*).
- Todos los atributos relevantes según el enunciado, indicando si hay claves candidatas alternativas (por ejemplo, un atributo único que no sea la clave elegida).
- El tipo de cada atributo a nivel conceptual (texto, numérico, fecha, booleano), sin preocuparte todavía por el tipo exacto de SQL.

Y para cada relación entre entidades:

- Su cardinalidad (1:1, 1:N o N:M).

- La participación de cada entidad en la relación (total u obligatoria vs. parcial u opcional), justificada con la regla de negocio correspondiente.

Preguntas guía (respondelas por escrito junto al diagrama, no solo en el dibujo):

- ¿Por qué la relación entre producto y pedido no puede resolverse como una relación binaria simple 1:N? ¿Qué información se perdería si lo intentaras?
- ¿Qué entidad tiene participación parcial en la relación con categoría, y cuál tiene participación total? ¿Qué pasaría si invirtieras esa lectura?
- ¿Alguno de tus atributos podría descomponerse en partes más simples (por ejemplo, un nombre completo en nombre y apellido)? Justificá tu decisión de dejarlo compuesto o de separarlo.

Entregable: diagrama ER (imagen, PDF o enlace a la herramienta usada) + diccionario de entidades y atributos en una tabla + respuestas a las preguntas guía.

Parte 2 — Derivación al modelo relacional (20%)

Convertí tu modelo ER de la Parte 1 en un modelo relacional. Escribí cada tabla resultante con la notación tabla (atributo1, atributo2, ...), subrayando la clave primaria y marcando las claves foráneas con una flecha, por ejemplo:

```
producto (id, nombre, precio, stock, categoria_id -> categoria)
```

Tené en cuenta las siguientes reglas de derivación:

- Cada entidad fuerte del ER se convierte en una tabla, con su clave primaria.
- Una relación 1:N se resuelve agregando la clave primaria del lado "1" como clave foránea en la tabla del lado "N" (no se crea una tabla nueva para la relación).
- Una relación N:M se resuelve creando una tabla intermedia (entidad asociativa) que contiene, como mínimo, las claves foráneas hacia ambas entidades participantes; si la relación tiene atributos propios (por ejemplo, cantidad), esos atributos viven en la tabla intermedia.
- Decidí si la clave primaria de cada tabla intermedia es la combinación de las dos claves foráneas (clave compuesta) o una clave sustituta (surrogate key) propia, y justificá tu elección.

Preguntas guía:

- ¿Qué pasaría con la integridad de tus datos si la tabla intermedia no incluyera ambas claves foráneas como NOT NULL?
- Si el pedido pudiera existir sin ningún producto asociado (por ejemplo, un pedido recién iniciado), ¿cambia esto la participación que definiste en la Parte 1 para la relación con la tabla intermedia? Explica el impacto.

Entregable: listado completo de tablas derivadas, con clave primaria, claves foráneas y atributos, más las respuestas a las preguntas guía.

Parte 3 — Normalización hasta 3FN/BCNF (25%)

Food Store todavía registra sus ventas en una única planilla plana. Este es un extracto real de esa planilla (una fila por cada línea de producto vendido dentro de un pedido):

N.º pedido	Fecha	Cliente	Producto	Categoría	Precio unitario	Cant.	Subtotal	Forma de pago
1	01/03/2026	Ana Gómez	Muzzarella	Pizzas	1000.00	2	2000.00	EFFECTIVO
1	01/03/2026	Ana Gómez	Coca 1.5L	Bebidas	800.00	1	800.00	EFFECTIVO
2	01/03/2026	Luis Paz	Napolitana	Pizzas	1500.00	1	1500.00	TARJETA
3	05/03/2026	Ana Gómez	Muzzarella	Pizzas	1050.00	3	3150.00	TRANSFERENCIA
4	06/03/2026	Marta Ruiz	Coca 1.5L	Bebidas	800.00	4	3200.00	EFFECTIVO
4	06/03/2026	Marta Ruiz	Napolitana	Pizzas	1500.00	2	3000.00	EFFECTIVO
5	07/03/2026	Luis Paz	Muzzarella	Pizzas	1050.00	1	1050.00	TARJETA

Para este ejercicio, asumí que el nombre del cliente y el nombre del producto identifican de manera única a cada uno dentro de esta muestra (en la base de datos real vas a usar claves numéricas, como en las Partes 1, 2 y 4, pero para razonar sobre dependencias funcionales alcanza con trabajar sobre estos nombres).

Además de lo que se observa en los datos, valen estas reglas de negocio ya presentadas en la Sección 2:

- Cada pedido tiene una única fecha, un único cliente y una única forma de pago.
- Cada producto pertenece a una única categoría.
- Dentro de un mismo pedido, un producto no se repite en más de una línea.
- El precio de un producto puede variar en el tiempo (mira lo que pasa con la Muzzarella entre el pedido 1 y el pedido 3).

Pista: fijate que el precio de la Muzzarella cambia de un pedido a otro. Esa observación es la clave para decidir de qué depende realmente el atributo precio_unitario en esta planilla: ¿del producto solo, o de algo más?

Resuelve, mostrando el razonamiento paso a paso (no alcanza con enunciar el resultado final):

1. Determina la clave (o claves candidatas) de esta relación universal, a partir de qué combinación de atributos identifica una fila de forma única.
2. Lista todas las dependencias funcionales que puedas identificar entre los atributos de la planilla, apoyándote en los datos y en las reglas de negocio.
3. Verifica si la relación está en 1FN. Si no lo está, corregirlo.
4. Verifica 2FN: identifica si hay atributos que dependen solo de una parte de la clave (dependencia parcial) y, si los hay, sepáralos en una tabla propia.
5. Verifica 3FN: identifica si, luego de resolver 2FN, queda alguna dependencia transitiva entre atributos no clave, y resolverla.
6. Verifica BCNF: para cada tabla resultante, confirmá que todo determinante (todo atributo o conjunto de atributos que determina a otro) es una clave candidata. Si encontrás una excepción, corregila; si no la encontrás, decilo explícitamente y por qué.
7. Presentá el conjunto final de tablas normalizadas, con su clave primaria marcada.

Pregunta de integración (para responder después de terminar la normalización):

- Comparé las tablas que obtuviste normalizando la planilla con las tablas que derivaste en la Parte 2 a partir de tu modelo ER. ¿A qué entidad de tu ER corresponde cada tabla? ¿Hay atributos de tu ER (por ejemplo, el mail del cliente) que no aparecían en esta planilla? ¿Qué te dice eso sobre la diferencia entre modelar a partir de un enunciado de negocio y normalizar a partir de datos históricos ya existentes?

- El atributo subtotal de cada línea es, en teoría, calculable como cantidad × precio_unitario. ¿Conviene almacenarlo igual, o siempre recalcularlo al vuelo? ¿Esa decisión de diseño viola alguna forma normal, o es un tema distinto (rendimiento, trazabilidad histórica)? Justificá en dos o tres líneas.

Entregable: desarrollo escrito de los siete pasos, con las dependencias funcionales notadas explícitamente (por ejemplo, nro_pedido → fecha, cliente, forma_pago) y las tablas finales resultantes, más las respuestas a las preguntas de integración.

Parte 4 — DDL en PostgreSQL (25%)

Escribí el script **schema.sql** que crea en PostgreSQL el esquema definitivo — el que obtuviste al conciliar los resultados de las Partes 2 y 3. El script debe ejecutarse sin errores de principio a fin.

Requisitos mínimos que tu script debe cumplir:

- Tipo de dato adecuado para cada atributo: identificadores numéricos con clave autogenerada, textos con longitud máxima razonable, montos monetarios con precisión fija (no de punto flotante), cantidades enteras, valores booleanos, fechas/horas con zona horaria, y tipos enumerados (CREATE TYPE ... AS ENUM) para los dominios cerrados que identificaste (por ejemplo, forma de pago).
- Clave primaria explícita en cada tabla.
- Clave foránea explícita (REFERENCES) en cada relación 1:N y en la tabla intermedia de la relación N:M, decidiendo y justificando el comportamiento ante un borrado del lado referenciado (RESTRICT, o alguna alternativa que prefieras).
- Al menos una restricción UNIQUE que refleje una clave candidata identificada en la Parte 1 (por ejemplo, el mail del cliente).
- Al menos tres restricciones CHECK que expresen reglas de negocio del enunciado (por ejemplo, precio y stock no negativos).
- NOT NULL en todo atributo cuya entidad tenga participación total en la relación correspondiente, según lo que definiste en la Parte 1; nutable en los que tengan participación parcial.
- DEFAULT donde corresponda (por ejemplo, marca de baja lógica en falso, fecha de creación en el momento de la inserción).
- Al menos dos índices, cada uno acompañado de una línea de comentario que justifique qué consulta esperada acelera (por ejemplo, buscar los pedidos de un cliente, o listar productos vigentes de una categoría).

Formato esperado (el ejemplo usa un dominio genérico, no forma parte de la resolución):

```
CREATE TABLE pais (  
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  nombre      VARCHAR(80) NOT NULL UNIQUE,  
  codigo_iso  VARCHAR(3)  NOT NULL,  
  activo      BOOLEAN     NOT NULL DEFAULT TRUE,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

Entregable: archivo **schema.sql** completo y ejecutable, con comentarios breves donde una decisión de diseño no sea obvia (por ejemplo, por qué elegiste `ON DELETE RESTRICT` en una FK puntual).

4. Formato y modalidad de entrega

- Modalidad: [a definir por la cátedra: individual o en grupos de hasta N integrantes].

- Entregables en un único archivo comprimido (.zip): diagrama ER (imagen o PDF), documento con el desarrollo de las Partes 2 y 3 (PDF o Word), y el archivo schema.sql de la Parte 4.
- Nombrá el archivo comprimido como Apellido_Nombre_TP1.zip (o Grupo_N_TP1.zip si es en grupo).
- Vía de entrega y fecha límite: [completar según el aula virtual de la cátedra].
- Se espera trabajo original. Se puede discutir el enfoque con compañeros, pero cada entrega (o grupo) debe redactar y resolver su propia versión.

5. Criterios de evaluación

La nota final del TP se compone según la siguiente rúbrica:

Criterio	Qué se evalúa	%
1. Modelo ER	Entidades y atributos correctos; claves primarias y candidatas bien identificadas; cardinalidades y participación (total/parcial) correctamente representadas y justificadas.	25
2. Modelo relacional	Cada entidad y relación del ER se traduce correctamente; las relaciones 1:N usan clave foránea en el lado N; la relación N:M se resuelve con tabla intermedia con su propia clave; tipos y nulabilidad coherentes con el ER.	20
3. Normalización	Dependencias funcionales identificadas correctamente; pasos 1FN → 2FN → 3FN/BCNF desarrollados y justificados (no solo enunciados); verificación explícita de BCNF.	25
4. DDL	Tipos de datos adecuados; PK y FK correctas; restricciones declarativas (NOT NULL, UNIQUE, CHECK, DEFAULT); al menos dos índices justificados; script ejecutable sin errores de sintaxis.	25
5. Integración y reflexión	Claridad y prolijidad de la presentación; respuestas fundamentadas a las preguntas de integración planteadas en la consigna.	5
Total		100

6. Recursos recomendados

- Herramientas de diagramación ER: dbdiagram.io, draw.io / diagrams.net, Lucidchart, o papel y lápiz escaneado — cualquiera es válida mientras el diagrama sea legible.
- Motor de base de datos: PostgreSQL (local, en un contenedor, o en un servicio como db-fiddle.com) con un cliente como pgAdmin, DBeaver o la terminal psql, para probar que tu schema.sql se ejecuta sin errores.
- Material teórico visto en clase sobre modelo entidad-relación, reglas de derivación al modelo relacional, formas normales y sintaxis DDL de PostgreSQL.

7. La IA como motor primario en este TP

Como en todas las cátedras de la carrera, en este TP la inteligencia artificial no es una herramienta opcional de apoyo: es el motor primario con el que se produce el trabajo, con OpenCode y Kiro como herramientas transversales del segundo año. Esto no reemplaza tu criterio como responsable de cada script y cada decisión de diseño: lo pone a prueba en cada paso.

La IA como motor primario

- Puesta a punto del flujo del segundo año sobre datos: OpenCode y Kiro sobre un repositorio con los scripts de la base, y el protocolo de seguridad de la cátedra: copia, transacción, respaldo.
- Escenarios de concurrencia reconstruidos con IA: dado un conflicto entre transacciones, explicar qué pasó y qué nivel de aislamiento lo evita, verificando en el motor.
- El riesgo fundacional: por qué un script generado jamás se ejecuta sin leerse; casos reales de destrucción de datos por automatización acrítica.

Actividades prácticas con IA

- Laboratorio: reproducir anomalías de aislamiento con dos sesiones concurrentes sobre la base del proyecto; informe explicado con apoyo de IA y verificado.
- Primer ejercicio con OpenCode: scripts de restricciones e integridad versionados en Git, con revisión de diff y defensa.

Nota: los dos primeros puntos de esta sección (concurrencia y transacciones) anticipan contenidos que se profundizan en la Semana 2 de la cátedra; en este TP alcanza con dejar registrada la puesta a punto del flujo de IA y el primer ejercicio de integridad versionado en Git. Documentá todo en tu Declaración de Uso de IA (DUIA): herramienta usada, para qué, y con qué prompts o specs.

Anexo I — Glosario rápido

- **Entidad:** objeto o concepto del dominio sobre el que se necesita almacenar información (por ejemplo, producto).
- **Atributo:** propiedad o característica de una entidad (por ejemplo, precio).
- **Clave candidata:** conjunto mínimo de atributos que identifica de forma única a cada instancia de una entidad; puede haber más de una.
- **Clave primaria:** la clave candidata elegida para identificar cada fila de una tabla.
- **Clave foránea:** atributo (o conjunto de atributos) de una tabla que referencia la clave primaria de otra, materializando una relación.
- **Cardinalidad:** cuántas instancias de una entidad pueden asociarse con cuántas instancias de otra en una relación (1:1, 1:N, N:M).
- **Participación total / parcial:** indica si toda instancia de una entidad debe obligatoriamente participar en una relación (total) o si puede existir sin participar (parcial).
- **Dependencia funcional (FD):** notación $A \rightarrow B$ que expresa que el valor de A determina de forma única el valor de B.
- **1FN:** todos los atributos tienen valores atómicos (sin listas ni grupos repetidos dentro de una misma celda).
- **2FN:** está en 1FN y, además, ningún atributo no clave depende de solo una parte de una clave compuesta.

- **3FN:** está en 2FN y, además, ningún atributo no clave depende transitivamente de la clave (a través de otro atributo no clave).
- **BCNF:** para toda dependencia funcional no trivial $X \rightarrow Y$, X es una clave candidata (una condición más estricta que 3FN).